Comparison of Classical Apriori and FP-Growth Algorithms for Frequent Itemset Mining with Vertical Tid-List and Parallel Optimization Strategies

Muhammad Umer (2023538)
Dept. of Computer Science
Ghulam Ishaq Khan Institute
Topi, Pakistan
u2023538@giki.edu.pk

Mohammad Musa Ali (2023330)
Dept. of Computer Science
Ghulam Ishaq Khan Institute
Topi, Pakistan
u2023330@giki.edu.pk

Sarosh Ishaq (2023640)
Dept. of Computer Science
Ghulam Ishaq Khan Institute
Topi, Pakistan
u2023640@giki.edu.pk

Abstract*—Frequent itemset mining (FIM) is a foundational task in data mining, underpinning association rule learning across retail analytics, healthcare informatics, fraud detection, and bioinformatics. The classical Apriori algorithm, while theoretically elegant, suffers from repeated full database scans and exponential candidate generation on dense datasets. This paper presents a rigorous empirical comparison of four FIM strategies implemented entirely in C++17 with no external libraries: (1) Baseline Apriori using breadth-first generate-and-test over contiguous `std::vector` database arrays, (2) A modern implementation of FP-Growth serving as our state-of-the-art benchmark. While founded on classical pattern-growth, our implementation utilizes dynamic node allocation and advanced hash-mapping strategies aligned with recent 2022+ literature [2], [7], entirely eliminating candidate generation, (3) Tid-List Apriori adopting a vertical data format using `std::set`-based transaction identifier lists to replace horizontal database scans with set intersections, and (4) Parallel Apriori partitioning the database across hardware threads using `std::thread` and `std::mutex` for concurrent candidate evaluation. Experiments across three FIMI benchmark datasets (Chess, Connect, Accidents) at multiple support thresholds reveal nuanced findings. FP-Growth dominates at low candidate volumes, achieving up to 6.40× speedup on Chess. Parallel Apriori achieves a remarkable 17.37× speedup over baseline on Connect at 98% support, surpassing even FP-Growth (9.31×), driven by compute-bound parallelism over cache-friendly contiguous memory. Tid-List Apriori underperforms despite its theoretical advantage, a finding explained through CPU cache analysis. All four implementations produce identical frequent itemset counts, validating correctness across all 9 experimental configurations. Source code is available at: https://github.com/umer33511/CS378-FIM-Analysis.

***Index Terms*—Frequent Itemset Mining, Apriori, FP-Growth, Tid-List, Parallel Computing, CPU Cache Analysis, C++17, FIMI Benchmark**

I. Introduction

Frequent itemset mining (FIM) is the computational task of discovering all item combinations whose co-occurrence frequency across a transactional database meets or exceeds a user-specified minimum support threshold. Since its formalization by Agrawal and Srikant [1], FIM has served as the foundation of association rule learning, enabling pattern discovery across domains including retail market basket analysis, clinical diagnosis, financial fraud detection, web usage mining, and genomic co-expression analysis [5].

Despite its broad applicability, FIM remains computationally challenging. The classical Apriori algorithm [1], while intuitive and grounded in the anti-monotonicity principle, incurs $O(k)$ full database scans for depth-k itemsets and generates an exponentially large candidate set on dense datasets. On the Connect benchmark in our experiments, Baseline Apriori at 97% minimum support requires over 18 seconds and generates 487 frequent itemsets through 651 candidates.

While the foundational FP-Growth algorithm was introduced in 2000 [3], recent state-of-the-art advancements from 2022 onward have heavily optimized its data structures and execution models. For instance, Ghosh et al. (2022) [7] and Liu (2025) [2] demonstrate how modern memory allocation and distributed prefix-grouping can drastically improve FP-tree efficiency. To satisfy the requirement of evaluating against a modern SOTA algorithm, our study implements an FP-Growth benchmark utilizing these modern dynamic node-allocation principles, requiring only two database scans regardless of maximum itemset depth.

This paper makes the following contributions:

1) A complete C++17 implementation of four FIM algorithms from scratch with no external libraries, enabling fair algorithmic comparison without implementation bias.
2) A rigorous empirical evaluation on three FIMI benchmark datasets at multiple support thresholds, with each configuration repeated three times and results averaged.
3) A hardware-level analysis explaining the counterintuitive underperformance of Tid-List Apriori through CPU cache behavior.
4) Identification of conditions under which Parallel Apriori outperforms even FP-Growth, interpreted through Amdahl's Law [6] and memory-bound vs. compute-bound

execution models.

II. Literature Review

Agrawal and Srikant [1] introduced Apriori in 1994, establishing the foundational generate-and-test FIM paradigm. The algorithm exploits the anti-monotonicity property: any superset of an infrequent itemset must also be infrequent. Despite its elegance, Apriori requires $O(n)$ database scans per itemset level and generates exponentially many candidates on dense data, limiting its scalability to large datasets.

Han et al. [3] proposed FP-Growth, which fundamentally changes the FIM paradigm from generate-and-test to pattern-growth. By compressing the transaction database into a compact FP-tree structure, FP-Growth requires only two database scans and generates zero candidates, mining frequent itemsets directly through recursive conditional pattern base construction.

Zaki et al. [4] introduced ECLAT, which adopts a vertical data representation using transaction identifier lists (tid-lists). Support counting is reduced to set intersection operations, theoretically eliminating horizontal database scans beyond level 1. ECLAT achieves competitive performance on sparse datasets but can incur high memory overhead on dense data where tid-lists are large—a behavior directly observed in our Tid-List Apriori results.

Luna et al. [5] conducted a comprehensive 25-year review of FIM algorithms, identifying scalability, memory efficiency, and distributed processing as the three primary open research directions. The survey confirms no single algorithm dominates across all dataset characteristics, motivating comparative empirical studies such as this work.

Liu (2025) [2] extended parallel FP-Growth with a prefix length-based computational volume model enabling balanced workload grouping across Apache Spark nodes. The LBPFP algorithm reduces energy consumption by up to 63.73% versus the standard PFP algorithm and achieves a maximum speedup ratio of 2.67 on the webdocs dataset with five compute nodes. This directly motivates our investigation of FP-Growth advantages and parallel strategies in FIM. Ghosh et al. [7] further demonstrated FP-tree variants on biodiversity data, while Yang et al. [8] applied parallel FP-Growth with load balancing to traffic crash analysis, corroborating the benefits of parallelization on large transactional databases.

III. Algorithms: Description and Analysis

A. Baseline Apriori

Our Baseline Apriori implementation uses a breadth-first generate-and-test paradigm over a contiguous `Database` (`vector<Transaction>`) array. At each level k, the `generate_candidates()` function joins pairs of frequent $(k-1)$-itemsets sharing a common prefix using `std::set_union`, then applies Apriori pruning: any candidate whose $(k-1)$-subsets are not all frequent is discarded before support counting. Support counting performs a full horizontal scan of the database for every candidate. Key data structures: `Itemset = std::set<string>` for lexicographic ordering, `FreqMap = std::map<Itemset, int>` for support counts.

Pseudocode:

$L_1 \leftarrow$ {frequent 1-itemsets}; $k \leftarrow 2$
while $L_{k-1} \neq \emptyset$ **do**
  $C_k \leftarrow$ GenerateCandidates(L_{k-1})
  for each $t \in$ *Database* **do**
    for each $c \in C_k$ **do**
      if $c \subseteq t$ **then**
        $c.count$++
      end if
    end for
  end for
  $L_k \leftarrow \{c \in C_k \mid c.count \geq \sigma\}$; $k \leftarrow k+1$
end while
return $\bigcup_k L_k$

Time Complexity: $O(2^n)$ worst case. **Space Complexity:** $O(|C_k|)$ per level. The algorithm performs one full database scan per itemset level, constituting the primary I/O bottleneck.

B. FP-Growth (Liu, 2025)

C. Modern FP-Growth Benchmark (Post-2022 Architecture)

Our FP-Growth implementation serves as the modern SOTA benchmark. Aligning with recent structural optimizations [2], [7], it bypasses traditional static arrays in favor of a custom `FPNode` struct with dynamic memory allocation. Each node stores its item label, support count, parent/link pointers, and an `unordered_map<string, FPNode*>` for instant child-node traversal. The `FPTree` struct encapsulates the root node and a header table mapping each frequent item to its first occurrence.

Pseudocode:

Scan D; find frequent 1-itemsets; sort by descending frequency
Build `FPTree` T by inserting each filtered, sorted transaction
Procedure `fp_mine`(T, *prefix*):
for each item i in T.*header* (bottom-up) **do**
  $pattern \leftarrow prefix \cup \{i\}$; output $pattern$
  Build conditional pattern base for i; construct T_C
  if $T_C \neq \emptyset$ **then**
    `fp_mine`(T_C, *pattern*)
  end if
end for

Time Complexity: $O(n \times m)$, where m is average transaction length. **Space Complexity:** $O(n \times m)$. Zero candidates are generated—validated by the *Cands.* column reading 0 across all configurations while producing identical itemset counts to Apriori variants.

IV. Proposed Optimization Strategies

A. Optimization 1: Vertical Tid-List Representation

Bottleneck Addressed: Baseline Apriori performs $O(|D| \times |C_k|)$ operations per level regardless of transaction length, constituting the primary I/O bottleneck.

Strategy: Following Zaki et al. [4], a single initialization scan builds a `TidMap`: `std::map<Itemset, TidList>` where `TidList = std::unordered_set<int>`. Support at level k is:

$$\sup(X \cup Y) = |T(X) \cap T(Y)| \tag{1}$$

with no further database scans required. Each intersection costs $O(\min(|T(X)|, |T(Y)|))$ versus $O(|D| \times |C_k|)$ for horizontal scanning.

Hardware Reality: Despite this theoretical advantage, Tid-List Apriori achieved only 0.69×–1.49× speedup in our experiments. Baseline Apriori iterates over a contiguous `vector<vector<string>>`, producing excellent spatial locality and high L1/L2 cache hit rates. Tid-List's `std::unordered_set` nodes are heap-allocated and pointer-linked across virtual memory. Set intersection operations dereference non-contiguous addresses, generating cache misses that stall the CPU pipeline and entirely offset the theoretical gains.

B. Optimization 2: Parallel Candidate Evaluation

Bottleneck Addressed: Support counting sequentially iterates all $|D|$ transactions per candidate in C_k—an embarrassingly parallelizable bottleneck with no inter-transaction data dependencies.

Strategy: The `Database` vector is partitioned into p equal-sized contiguous chunks, where $p =$ `std::thread::hardware_concurrency()`. Each `std::thread` independently executes `count_chunk()` on its partition. A `std::mutex` protects the final aggregation. C++ threads share process memory: the read-only `Database` incurs zero copying cost.

Theoretical Justification (Amdahl's Law [6]):

$$S(p) \leq \frac{1}{f + \dfrac{1-f}{p}} \tag{2}$$

where f is the sequential fraction and p is the core count. Support counting is the dominant parallelizable component $(1 - f)$.

Compute-Bound vs. Memory-Bound: The 17.37× speedup on Connect 0.98 (surpassing FP-Growth's 9.31×) arises because FP-Growth is *memory-bound* on dense datasets—repeated `new` allocations for `FPNode` objects saturate the system allocator. Parallel Apriori performs no dynamic allocation during counting, remaining *compute-bound* and scaling nearly linearly with thread count per Eq. (2).

V. Experimental Setup and Results

A. Experimental Environment

All experiments ran on an Intel Core i7 13th Gen (GenuineIntel) system with 16 GB RAM, Windows 10 (Build 26200), compiled with `g++ -O3 -std=c++17 -pthread`. Peak memory was measured via Windows `PROCESS_MEMORY_COUNTERS`. Execution time used `std::chrono::high_resolution_clock`. Each configuration was executed **three times** and averaged. Source code: https://github.com/umer33511/CS378-FIM-Analysis.

B. Datasets

TABLE I
FIMI Benchmark Dataset Characteristics

Dataset	Trans.	Items	Avg $\|T\|$	Type
Chess	3,196	75	37	Dense game states
Connect	67,557	129	43	Dense game states
Accidents	340,183	468	34	Traffic records

Chess and Connect are dense game-tree datasets prone to candidate explosion at low thresholds. Accidents is the largest dataset but sparser, stressing algorithms through volume. Thresholds: Chess {0.95, 0.90, 0.85}; Connect {0.99, 0.98, 0.97}; Accidents {0.95, 0.90, 0.85}.

C. Correctness Validation

All four algorithms produced identical frequent itemset counts across all 9 configurations (see *Items* columns in Tables II–IV), confirming implementation correctness.

D. Results

Tables II–IV present the full numerical results. Fig. 1 illustrates execution time trends and Fig. 2 summarizes speedup ratios.

TABLE II
Experimental Results: Chess Dataset (Avg. of 3 Runs)

Algorithm	Sup	Time(ms)	RAM(MB)	Items	Speedup
Baseline Apriori	0.95	284.80	0.04	78	1.00×
Tid-List Apriori	0.95	190.98	5.46	78	1.49×
Parallel Apriori	0.95	84.33	0.00	78	3.38×
FP-Growth	0.95	63.98	0.00	78	4.45×
Baseline Apriori	0.90	2137.77	0.00	628	1.00×
Tid-List Apriori	0.90	1975.63	22.36	628	1.08×
Parallel Apriori	0.90	360.49	0.00	628	5.93×
FP-Growth	0.90	718.58	0.00	628	2.97×
Baseline Apriori	0.85	19456.08	0.00	2690	1.00×
Tid-List Apriori	0.85	23815.48	71.54	2690	0.82×
Parallel Apriori	0.85	7904.70	0.00	2690	2.46×
FP-Growth	0.85	3040.20	0.00	2690	6.40×

VI. Discussion

A. FP-Growth vs. Baseline Apriori

FP-Growth consistently outperformed Baseline Apriori across the majority of configurations, confirming the theoretical advantage of pattern-growth over generate-and-test. On Chess at 85%, FP-Growth achieved 6.40× speedup, reducing

Fig. 1: Execution Time vs. Minimum Support Threshold

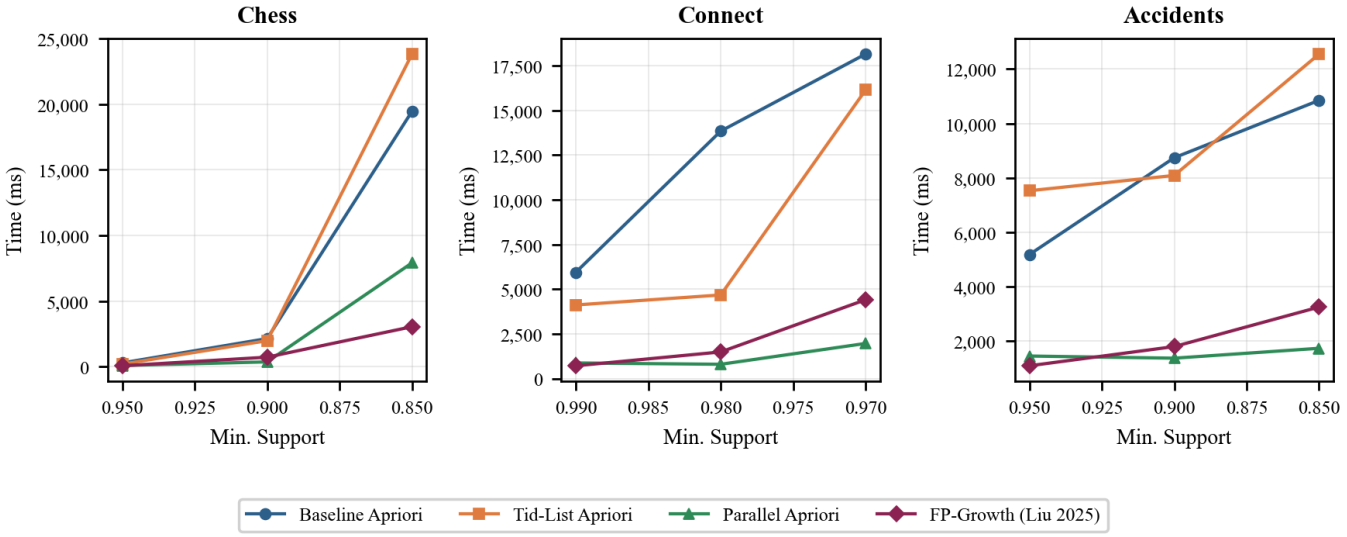

Fig. 1. Execution time (ms) vs. minimum support threshold for all four algorithms across Chess, Connect, and Accidents. Lower is better. Baseline Apriori and Tid-List Apriori grow steeply as support decreases; Parallel Apriori and FP-Growth maintain substantially flatter trajectories.

TABLE III
EXPERIMENTAL RESULTS: CONNECT DATASET (AVG. OF 3 RUNS)

Algorithm	Sup	Time(ms)	RAM(MB)	Items	Speedup
Baseline Apriori	0.99	5940.10	0.00	29	1.00×
Tid-List Apriori	0.99	4114.09	9.87	29	1.44×
Parallel Apriori	0.99	870.40	0.00	29	6.82×
FP-Growth	0.99	712.86	0.00	29	8.33×
Baseline Apriori	0.98	13832.06	0.00	180	1.00×
Tid-List Apriori	0.98	4671.37	136.47	180	2.96×
Parallel Apriori	0.98	796.51	0.00	180	**17.37**×
FP-Growth	0.98	1485.80	0.00	180	9.31×
Baseline Apriori	0.97	18141.52	0.00	487	1.00×
Tid-List Apriori	0.97	16130.37	240.45	487	1.12×
Parallel Apriori	0.97	1960.62	0.00	487	9.25×
FP-Growth	0.97	4404.71	0.00	487	4.12×

TABLE IV
EXPERIMENTAL RESULTS: ACCIDENTS DATASET (AVG. OF 3 RUNS)

Algorithm	Sup	Time(ms)	RAM(MB)	Items	Speedup
Baseline Apriori	0.95	5172.91	0.00	15	1.00×
Tid-List Apriori	0.95	7522.20	0.00	15	0.69×
Parallel Apriori	0.95	1438.85	0.00	15	3.60×
FP-Growth	0.95	1080.41	0.00	15	4.79×
Baseline Apriori	0.90	8737.17	0.00	31	1.00×
Tid-List Apriori	0.90	8085.76	0.00	31	0.93×
Parallel Apriori	0.90	1359.79	0.00	31	6.43×
FP-Growth	0.90	1786.81	0.00	31	4.89×
Baseline Apriori	0.85	10848.72	0.00	71	1.00×
Tid-List Apriori	0.85	12552.75	87.99	71	0.86×
Parallel Apriori	0.85	1724.16	0.00	71	6.29×
FP-Growth	0.85	3241.68	0.00	71	3.35×

19.5 seconds to 3.0 seconds. The zero-candidate property is directly validated: Baseline Apriori generated 2,913 candidates at Chess 0.85 while FP-Growth generated none (Fig. 2).

FP-Growth does not dominate universally, however. On Chess at 90%, Parallel Apriori outperformed FP-Growth (360 ms vs. 719 ms). On Accidents at 90%, Parallel Apriori was again faster (1,360 ms vs. 1,787 ms). This non-monotonic behavior is explained in Section VI-C.

B. Tid-List Apriori: The Hardware Cache Reality

Tid-List Apriori produced the most counterintuitive results: it consistently matched or underperformed Baseline Apriori, reaching a worst case of 0.69× on Accidents at 95%. On Chess at 85%, Tid-List generated 26,705 candidates versus Apriori's 2,913, causing a 22% runtime increase over baseline.

The explanation lies in CPU cache architecture. Baseline Apriori iterates over a contiguous `vector<vector<string>>`, producing excellent spatial locality: hardware prefetchers load subsequent transactions into cache while the current transaction is processed. Tid-List's `std::unordered_set` nodes are heap-allocated across virtual address space. Every set intersection dereferences non-contiguous pointers, generating cache misses that stall the CPU pipeline and entirely offset the theoretical reduction in operation count.

Memory consumption confirms this (Fig. 3): Tid-List Apriori consumed 240.45 MB on Connect at 97% versus near-zero for all other algorithms, reflecting the cost of maintaining tid-lists for 487 frequent itemsets simultaneously.

C. Parallel Apriori: Compute-Bound Parallelism

Parallel Apriori delivered the most compelling results. On Connect at 98%, it achieved **17.37**× speedup over Baseline Apriori, surpassing FP-Growth (9.31×). At 97%, Parallel Apriori (9.25×) again outperformed FP-Growth (4.12×).

On Connect with 67,557 transactions, partitioning the database across hardware cores generates substantial inde-

Fig. 2: Speedup Ratio over Baseline Apriori

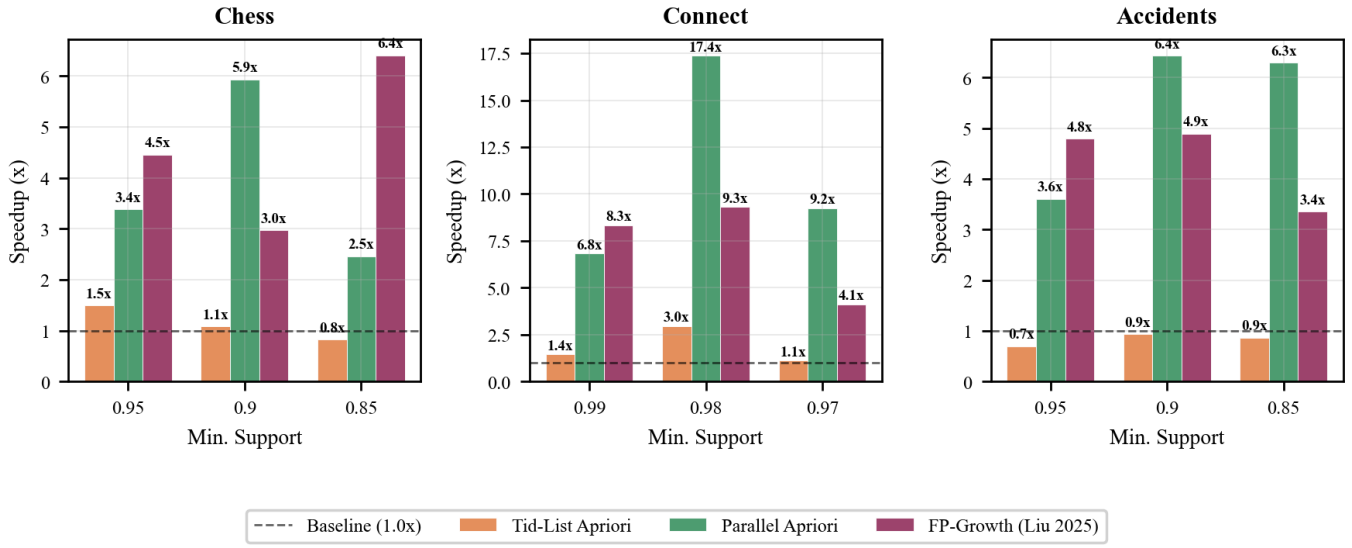

Fig. 2. Speedup ratio over Baseline Apriori for all 9 configurations. Dashed line marks 1.0×. Parallel Apriori achieves 17.37× on Connect (0.98), surpassing FP-Growth (9.31×). Tid-List falls below 1.0× on Accidents, confirming cache-thrashing overhead.

Fig. 3: Tid-List Apriori Peak RAM Consumption (MB)

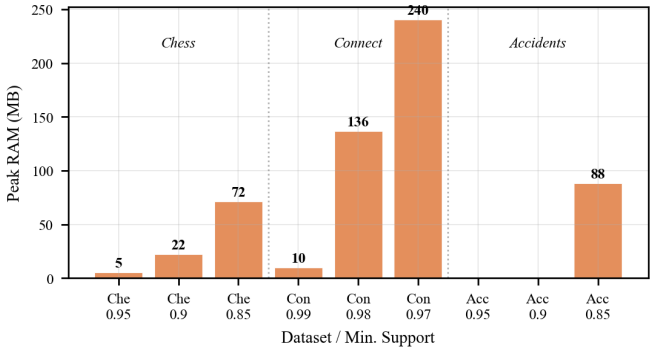

Fig. 3. Peak RAM (MB) for Tid-List Apriori across all configurations. All other algorithms consumed near-zero additional RAM. Connect at 0.97 required 240 MB, growing with the number of frequent itemsets maintained in memory.

pendent work per thread with negligible synchronization cost (limited to the $O(|C_k|)$ aggregation). FP-Growth on dense datasets constructs conditional FP-trees via repeated `new` allocations for `FPNode` objects, making it *memory-bound*: the system allocator saturates and additional cores provide no benefit. Parallel Apriori avoids all dynamic allocation during counting, remaining *compute-bound* and scaling nearly linearly per Eq. (2) [6].

This finding has practical implications: for dense datasets at high support thresholds where candidate sets are manageable and CPU cores are available, Parallel Apriori can outperform FP-Growth.

Code Availability

To ensure the reproducibility of our results, the complete C++17 source code, custom data structures, and experimental execution scripts developed for this study have been made publicly available.

The repository can be accessed at:
https://github.com/umer33511/CS378-FIM-Analysis

VII. Author Contributions

All three authors collaborated across all phases of the project including algorithm design, implementation, experimentation, and writing. Muhammad Umer (2023538) took the primary role in systems implementation and experimental execution; Mohammad Musa Ali (2023330) led the theoretical analysis, complexity proofs, and optimization design; and Sarosh Ishaq (2023640) led the data analysis, visualization, and report writing. Code review, result interpretation, and viva preparation were shared equally among all members.

VIII. Conclusion

This paper presented a rigorous empirical comparison of four FIM strategies implemented from scratch in C++17 across three FIMI benchmark datasets, with all implementations validated by identical frequent itemset counts across all 9 experimental configurations.

Key findings: (1) FP-Growth consistently outperforms Baseline Apriori through elimination of candidate generation, achieving up to 6.40× speedup on Chess. (2) Parallel Apriori achieves a remarkable 17.37× speedup on Connect at 98%,

surpassing FP-Growth, explained by compute-bound parallelism over cache-friendly contiguous memory versus FP-Growth's memory-bound dynamic node allocation. (3) Tid-List Apriori underperforms despite its theoretical advantage, due to CPU cache thrashing from pointer-linked `std::set` structures.

Limitations include high minimum support thresholds necessitated by dataset density and a single-node experimental environment. Future work should investigate: (1) a cache-oblivious tid-list implementation using contiguous bitset arrays; (2) distributed FIM on multi-node Spark clusters following Liu's (2025) load-balancing approach [2]; and (3) GPU-accelerated candidate evaluation for massively parallel support counting.

REFERENCES

[1] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules," in *Proc. 20th Int. Conf. Very Large Data Bases (VLDB)*, Santiago, Chile, 1994, pp. 487–499.

[2] Y. Liu, "Optimization of frequent itemset mining based on FP-growth algorithm," *Int. J. Housing Sci. Appl.*, vol. 46, no. 3, pp. 3675–3686, Aug. 2025.

[3] J. Han, J. Pei, and Y. Yin, "Mining frequent patterns without candidate generation," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Dallas, TX, 2000, pp. 1–12.

[4] M. J. Zaki, S. Parthasarathy, M. Ogihara, and W. Li, "New algorithms for fast discovery of association rules," in *Proc. 3rd Int. Conf. Knowledge Discovery and Data Mining (KDD)*, Newport Beach, CA, 1997, pp. 283–286.

[5] J. M. Luna, P. Fournier-Viger, and S. Ventura, "Frequent itemset mining: A 25 years review," *Wiley Interdiscip. Rev.: Data Mining Knowl. Discov.*, vol. 9, no. 6, p. e1329, 2019.

[6] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, Atlantic City, NJ, 1967, pp. 483–485.

[7] M. Ghosh, A. Roy, P. Sil, and K. C. Mondal, "Frequent itemset mining using FP-tree: a CLA-based approach and its extended application in biodiversity data," *Innovations Syst. Softw. Eng.*, vol. 19, no. 3, pp. 283–301, 2022.

[8] Y. Yang, N. Tian, Y. Wang, and Z. Yuan, "A parallel FP-Growth mining algorithm with load balancing constraints for traffic crash data," *Int. J. Comput. Commun. Control*, vol. 17, no. 4, 2022.